

COS 202: Computer Programming II

Advanced Object-Oriented Programming (Java)

Dr. Babatunde I. Ayinla

Faculty of Computing
University of Ibadan, Nigeria

July, 2026





Course Code: COS 202
Course Title: Advanced Object-Oriented Programming (Java)
Course Unit: 2
Lecturer: Dr. Babatunde I. Ayinla
Prepared: July, 2026

This course builds on COS 201. In the first two weeks we review Arrays and Methods to build a solid foundation for the new material (Polymorphism, Abstraction, Interfaces, Collections, Exceptions, GU



- Polymorphism
- Abstract classes and interfaces
- Class hierarchies & packages/namespaces
- Use of API
- Iterators/Enumerators
- List, Stack, Queue (from API)
- Searching, Sorting, Recursion
- Event-driven programming
- Event-handling; propagation; exceptions
- GUI programming



Learning Outcomes

By the end of this course, students should be able to:

- 1 Develop solutions to problems using object-oriented programming
- 2 Use modules/packages/namespaces for program organization
- 3 Use APIs in writing applications
- 4 Apply divide-and-conquer to searching/sorting (iterative or recursive)
- 5 Explain exceptions and how to handle them
- 6 Write simple multithreading applications
- 7 Design and implement a simple GUI application



Week One

Introduction to Array & Method



What is an Array?

Definition

An **array** is a collection of elements of the *same data type* stored in contiguous memory locations, accessed using an index.

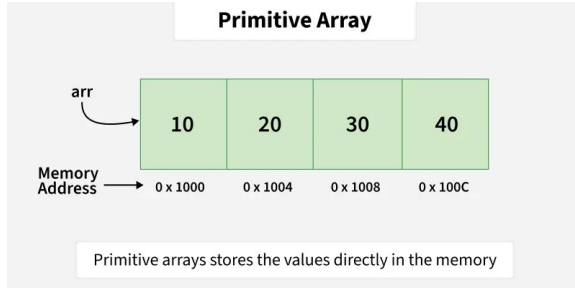


Figure: Primitive Array



- Holds **primitive types** (int, char, boolean, ...) and **objects** (String, Integer, ...)
- Primitive arrays store values in contiguous locations; non-primitive arrays store *references* in contiguous locations
- Once created, an array's **size is fixed** and cannot change



Primitive vs. Non-Primitive Arrays

```
public class Geeks {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40};           // primitive array  
        for (int i = 0; i < arr.length; i++)  
            System.out.print(arr[i] + " ");  
  
        String[] names = {"Babatunde", "Ayinla", "Oluwasemilore"}; // non-primitive  
        for (int i = 0; i < names.length; i++)  
            System.out.print(names[i] + " ");  
    }  
}
```

Output: 10 20 30 40 Lakshit Rahul Pankaj



Declaring and Initializing an Array

Declaration (only creates a reference):

```
dataType[] arrayName;    // preferred
dataType arrayName[];    // also legal
```

Initialization (memory allocated on the *heap* with *new*):

```
int arr[] = new int[size];
```

- Size is fixed once created (use `ArrayList`/`Vector` for growable collections)
- Elements auto-initialize: 0 (numeric), `false` (boolean), `null` (reference types)

Array literal (no *new* needed):

```
int[] arr = {1, 2, 3};    // length is inferred from the literal
```



Accessing & Updating Array Elements

Access — indexing starts at 0:

```
int[] arr = {2, 4, 8, 12, 16};  
System.out.print(arr[3] + " ");    // 12  
System.out.print(arr[0]);          // 2
```

Update — use the assignment operator:

```
arr[0] = 90;  
System.out.println(arr[0]);        // 90
```



Traversing an Array

Visit each element once, using a loop from 0 to length-1:

```
class Cos202Array {  
    public static void main(String[] args) {  
        int[] arr = {2, 4, 8, 12, 16};  
        for (int i = 0; i < arr.length; i++)  
            System.out.print(arr[i] + " ");  
    }  
}
```

Output: 2 4 8 12 16



Multi-Dimensional Arrays

- A **multi-dimensional array** is an array of arrays — stores data in tabular form (rows, columns): matrices, tables, grids
- The 2D array is the most commonly used form

	Column 0	Column 1	Column 2
Row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>

Figure: Multi-dimensional Array



Syntax:

```
data_type[dim1][dim2]...[dimN] name = new data_type[size1]...[sizeN];
```

Size: total elements = product of all dimension sizes.

Example

```
int[] [] [] x = new int[3][5][7]; stores  $3 \times 5 \times 7 = 105$  elements
```



2D Array Example

```
public class Cos202Mult {  
    public static void main(String[] args) {  
        int[][] arr;  
        arr = new int[1][3];        // 1 row, 3 columns  
        arr[0][0] = 3; arr[0][1] = 5; arr[0][2] = 7;  
        System.out.println("arr[0][0] = " + arr[0][0]);  
        System.out.println("arr[0][1] = " + arr[0][1]);  
        System.out.println("arr[0][2] = " + arr[0][2]);  
    }  
}
```

Output: arr[0][0]=3 arr[0][1]=5 arr[0][2]=7



2D Array: Rows, Columns & Literal Init

Syntax:

```
data_type[][] array_name = new data_type[x][y];  
array_name[row_index][col_index] = value;
```

A 2D array is viewed as a table with x rows (index $0 \dots x-1$) and y columns (index $0 \dots y-1$).

```
int[][] arr = { {1, 2}, {3, 4} };  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 2; j++)  
        System.out.print(arr[i][j] + " ");  
    System.out.println();  
}  
// Output:  1 2  
//          3 4
```



Section Takeaway: Arrays

Key Takeaway

- Arrays store fixed-size, same-typed data in contiguous memory; indexing starts at 0
- Declare with `[]`, allocate with `new`, or initialize directly with a literal
- Access/update elements via `arr[i]`; traverse with a loop over `arr.length`
- Multi-dimensional arrays (arrays of arrays) model matrices and tables; total size = product of dimensions



Week Two

Continuation of Method & Recursive



What is a Method?

Definition

A **method** is a block of code that only runs when it is called. Methods are also known as **functions**.

- You can pass data into a method as **parameters**
- Used to perform actions and **reuse code**: define once, use many times
- A method must be declared *within a class*



Creating and Calling a Method

```
public class Main {  
    static void myMethod() {                // method with no return value  
        System.out.println("I just got executed!");  
    }  
    public static void main(String[] args) {  
        myMethod();        // call it  
        myMethod();        // can be called many times  
        myMethod();  
    }  
}
```

- static: method belongs to the class, not an object
- void: method returns no value
- Call with the method name, parentheses, and a semicolon



Method Parameters and Arguments

Parameters act as variables inside the method; separate multiple parameters with commas.

```
public class Main {  
    static void myMethod(String fname) {  
        System.out.println(fname + " Ayinla");  
    }  
    public static void main(String[] args) {  
        myMethod("Babatunde");    // Babatunde Ayinla  
        myMethod("Oluwasemilore"); // Oluwasemilore Ayinla  
    }  
}
```

fname is the **parameter**; "Babatunde" etc. are the **arguments**.



Return Values

Use a data type (instead of void) and the return keyword:

```
public class Main {  
    static int myMethod(int x, int y) {  
        return x + y;  
    }  
    public static void main(String[] args) {  
        int z = myMethod(5, 3);    // store result in a variable  
        System.out.println(z);    // Outputs 8  
    }  
}
```



Method Overloading

Multiple methods can share the same name with **different parameters**:

```
static int plusMethod(int x, int y) {  
    return x + y;  
}  
static double plusMethod(double x, double y) {  
    return x + y;  
}  
public static void main(String[] args) {  
    int myNum1 = plusMethod(8, 5);           // int: 13  
    double myNum2 = plusMethod(4.3, 6.26);  // double: 10.56  
    System.out.println("int: " + myNum1);  
    System.out.println("double: " + myNum2);  
}
```



Key Takeaway

- Methods package reusable behaviour behind a name; they run only when called
- Parameters receive data (arguments) so a method can act on different inputs
- `void` methods return nothing; typed methods use `return` to send a value back
- Overloading lets the same method name handle different parameter lists — resolved at compile time



Definition

Recursion is the technique of a function calling *itself* to break a complicated problem into simpler sub-problems.

- Best understood by tracing through an example
- Every recursive method needs a **base case** to stop the calls



Recursion Example: Sum 1 to N

```
public class Main {  
    public static int sum(int k) {  
        if (k > 0) {  
            return k + sum(k - 1);  
        } else {  
            return 0;           // base case  
        }  
    }  
    public static void main(String[] args) {  
        int result = sum(10);  
        System.out.println(result); // 55  
    }  
}
```



Tracing the Recursion

When $\text{sum}(k)$ is called, it adds k to the sum of all numbers smaller than k . When k reaches 0, the method returns 0 and the calls stop unwinding:

$$10 + \text{sum}(9)$$

$$10 + (9 + \text{sum}(8))$$

$$10 + (9 + (8 + \text{sum}(7)))$$

$$\vdots$$

$$10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + \text{sum}(0)$$

$$10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0 = 55$$



Section Takeaway: Recursion

Key Takeaway

- Recursion solves a problem by calling the same method on a smaller version of it
- A **base case** is essential — without it, the recursion never terminates
- Each call is "suspended" until the deeper call returns, then results combine on the way back up



Week Three

Introduction to OOP; Constructors and Modifiers



Object-Oriented Programming (OOP)

Definition

OOP is a paradigm based on **objects** that bundle **data (fields)** and **behaviour (methods)**, closely modelling real-world entities.

Why OOP?

- Improves code reusability
- Enhances maintainability and scalability
- Makes programs easier to understand and manage
- Closely models real-world entities



Example: Car is a *class* (blueprint); BMW, Mercedes, Audi are *objects* (instances) of it.

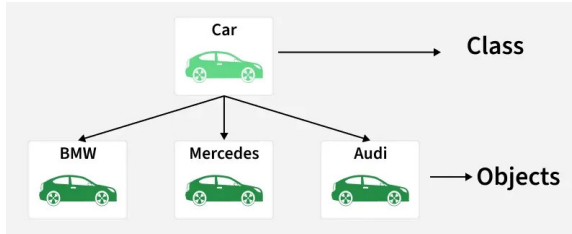


Figure: Class Description



Object

- An Object in Java is an instance of a class that represents a real-world entity. It is used to access the variables and methods defined inside a class.
- State: Represents the current data or attributes of an object.
- Behavior: Represents the actions that an object can perform.
- Identity: Every object has a unique identity in memory that distinguishes it from other objects, even if they contain the same data.



Example: Dog is a class, Tommy is an object of that class.

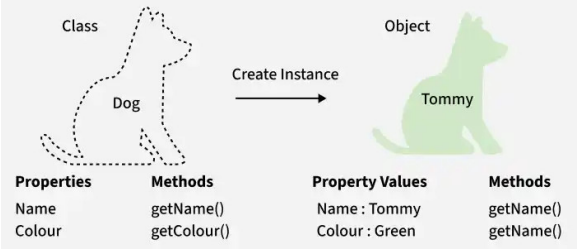


Figure: Class Description



Constructor

A **Constructor** is a special member of a class that is automatically invoked when an object is created. It is primarily used to initialize the object's data members and set up the initial state of an object.

- Constructor name must be the same as the class name.
- It does not have any return type, not even void.
- It is automatically called when an object is created using the new keyword.
- Constructors help initialize object properties during object creation.
- **Example:** When creating a Car object, a constructor can automatically assign values such as brand, model, and color.



Constructors at a Glance

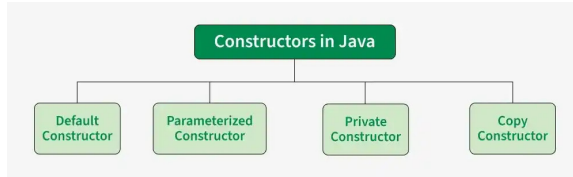


Figure: Type of Constructor



- **Default constructor:** no parameters; initializes with default values
- **Parameterized constructor:** accepts parameters to set specific values
- **Copy constructor:** creates a new object by copying another object's data
- **Private constructor:** `private` access; prevents creation from outside the class



- A constructor in Java is a special method that is used to initialize objects.
- The constructor is called when an object of a class is created.
- It can be used to set initial values for object attributes:



Constructor Example:

```
/ Create a Main class
public class Main {
    int x; // Create a class attribute

    // Create a class constructor for the Main class
    public Main() {
        x = 5; // Set the initial value for the class attribute x
    }

    public static void main(String[] args) {
        Main myObj = new Main(); // Create an object of class Main (This will call the constructor)
        System.out.println(myObj.x); // Print the value of x
    }
}

// Outputs 5
```



- Note that the constructor name must match the class name, and it cannot have a return type (like void).
- Also note that the constructor is called when the object is created.
- All classes have constructors by default: if you do not create a class constructor yourself, Java creates one for you.
- However, then you are not able to set initial values for object attributes.



- Constructors can also take parameters, which is used to initialize attributes. The following example adds an int y parameter to the constructor. Inside the constructor we set x to y ($x=y$). When we call the constructor, we pass a parameter to the constructor (5), which will set the value of x to 5:



Constructor Parameter:

```
public class Main {  
    int x;  
  
    public Main(int y) {  
        x = y;  
    }  
  
    public static void main(String[] args) {  
        Main myObj = new Main(5);  
        System.out.println(myObj.x);  
    }  
}  
  
// Outputs 5
```



Constructor Many Parameters:

```
public class Main {  
    int modelYear;  
    String modelName;  
  
    public Main(int year, String name) {  
        modelYear = year;  
        modelName = name;  
    }  
  
    public static void main(String[] args) {  
        Main myCar = new Main(2024, "ML350");  
        System.out.println(myCar.modelYear + " " + myCar.modelName);  
    }  
}  
  
// Outputs 2023 ML350
```



Modifiers By now, you are quite familiar with the `public` keyword that appears in almost all of our examples:

public class Main The `public` keyword is an access modifier, meaning that it is used to set the access level for classes, attributes, methods and constructors.

We divide modifiers into two groups:

- **Access Modifiers** - controls the access level
- **Non-Access Modifiers** - do not control access level, but provides other functionality



Table: Java Access Modifiers

Modifier	Description
public	The code is accessible for all classes
private	The code is only accessible within the declared class
default	The code is only accessible in the same package. This is used when you don't specify a modifier.
protected	The code is accessible in the same package and subclasses.



Public vs. Private Example

In the example below, the class has one public attribute and one private attribute.
Think of it like real life:

- **public** - a public park, everyone can enter
- **private** - your house key, only you can use it



Public vs. Private Example:

```
class Person {  
    public String name = "John";    // Public - accessible everywhere  
    private int age = 30;           // Private - only accessible inside this class  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        System.out.println(p.name);    // Works fine  
        System.out.println(p.age);    // Error: age has private access in Person  
    }  
}
```

Example explained Here, name is declared as public, so it can be accessed from outside the Person class. But age is declared as private, so it can only be used inside the Person class.



Week Four

Continuation of Access Modifiers



Final Example:

Non-access modifiers do not control visibility (like public or private), but instead add other features to classes, methods, and attributes.

The most commonly used non-access modifiers are **final**, **static**, and **abstract**. Final If you don't want the ability to override existing attribute values, declare attributes as final:

```
public class Main {  
    final int x = 10;  
    final double PI = 3.14;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        myObj.x = 50; // will generate an error: cannot assign a value to a final variable  
        myObj.PI = 25; // will generate an error: cannot assign a value to a final variable  
        System.out.println(myObj.x);  
    }  
}
```



Static Example:

A static method belongs to the class, not to any specific object. This means you can call it without creating an object of the class.

```
public class Main {  
    // Static method  
    static void myStaticMethod() {  
        System.out.println("Static methods can be called without creating objects");  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        myStaticMethod();           // Call the static method  
        Main.myStaticMethod();      // Or call it using the class name  
    }  
}
```

Note: A static method belongs to the class itself. You can call it without creating an object, but it cannot use variables or methods that belong to an object.



Abstract file:

An abstract method belongs to an abstract class, and it does not have a body. The body is provided by the subclass:

```
// abstract class
abstract class Main {
    public String fname = "John";
    public int age = 24;
    public abstract void study(); // abstract method
}

// Subclass (inherit from Main)
class Student extends Main {
    public int graduationYear = 2018;
    public void study() { // the body of the abstract method is provided here
        System.out.println("Studying all day long");
    }
}

/ Code from filename: Second.java
class Second {
    public static void main(String[] args) {
        // create an object of the Student class (which inherits attributes and methods from Main)
        Student myObj = new Student();

        System.out.println("Name: " + myObj.fname);
        System.out.println("Age: " + myObj.age);
        System.out.println("Graduation Year: " + myObj.graduationYear);
        myObj.study(); // call abstract method
    }
}
```

Non-Access Modifiers List

Non-Access Modifiers List For **classes**, you can use either **final** or **abstract**:

Table: Non-Access Modifiers

Modifier	Description
<code>final</code>	The class cannot be inherited by other classes (You will learn more about inheritance in the Inheritance chapter)
<code>abstract</code>	The class cannot be used to create objects (To access an abstract class, it must be inherited from another class. You will learn more about inheritance and abstraction in the Inheritance and Abstraction lessons)



Non-Access Modifiers

For **attributes** and methods, you can use the one of the following:

Table: Java Non-Access Modifiers

Modifier	Description
<code>final</code>	Attributes and methods cannot be overridden/modified
<code>static</code>	Attributes and methods belong to the class, not to objects. All objects share the same static attribute, and static methods can be called without creating objects.
<code>abstract</code>	Used only in an abstract class, and only on methods. The method has no body, e.g. <code>abstract void run();</code> . The body is provided by the subclass.
<code>transient</code>	Attributes and methods are skipped when serializing the object containing them
<code>synchronized</code>	Methods can only be accessed by one thread at a time
<code>volatile</code>	The value of an attribute is not cached thread-locally, and is always read from the “main memory”



Week Five

Characteristics of OOP



Definition

Hides implementation details and shows only **essential features** of an object — focus on *what* it does, not *how*.

- Hides complexity from the user
- Improves maintainability — internal changes don't affect user code
- Enhances flexibility via loose coupling
- Achieved using **abstract classes** (partial abstraction) and **interfaces** (100% abstraction)



Example: An ATM or a coffee machine represents abstraction, where the user interacts with simple operations while the internal working and implementation details remain hidden.

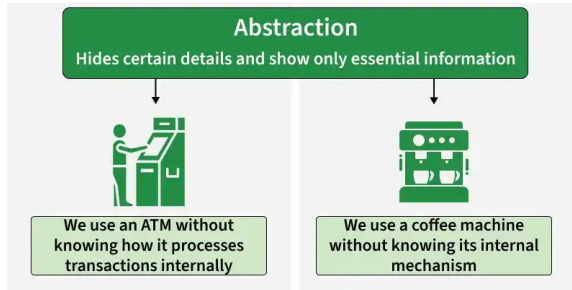


Figure: ATM Abstraction



How to Achieve Abstraction

- It is achieved in Java using abstract classes and interfaces. Interfaces can provide 100% abstraction, while abstract classes provide partial abstraction.

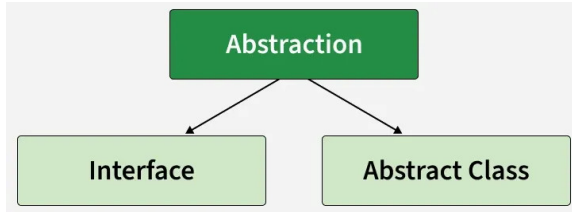


Figure: General Abstraction



Example: Abstract class defines a common base (Shape) using inheritance, while an interface (Drawable) defines behavior implemented by multiple classes.

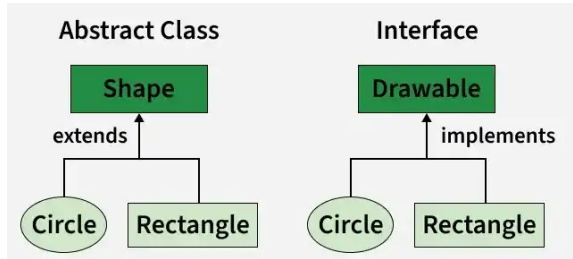


Figure: Interface Abstraction



Definition

Wraps data and methods into a single unit (a class) and **restricts direct access** to the data.

- Data members hidden using the `private` modifier
- Access provided through public **getter/setter** methods
- Improves data security, maintainability, and controlled access



Definition

Allows one class to acquire fields and methods of another using `extends`; represents an "is-a" relationship.

- **Superclass** is inherited from; **subclass** inherits (and may add its own features)
- **Single**: one subclass $\leftarrow$ one superclass
- **Multilevel**: a chain of derived classes
- **Hierarchical**: many subclasses $\leftarrow$ one superclass
- **Multiple** (via interfaces): Java has no multiple class inheritance
- **Hybrid** (via interfaces): combination of the above



Definition

"Many forms" — the same method/object shows **different behaviour** depending on context.

Example: `speak()` produces Bark, Meow, or Moo depending on whether the object is a Dog, Cat, or Cow.

Compile-time

(Method **Overloading**): same name, different parameters; resolved at compile time

Runtime

(Method **Overriding**): subclass redefines a superclass method; resolved at runtime



Section Takeaway: OOP Concepts

Key Takeaway

- OOP models software around objects that combine data and behaviour, improving reuse and maintainability
- **Abstraction** hides "how"; **Encapsulation** protects "what" (the data)
- **Inheritance** creates "is-a" relationships and reduces redundant code
- **Polymorphism** lets one interface take many forms — via overloading (compile-time) or overriding (runtime)



Practice Time!

Five exercises spanning arrays, methods, recursion, and OOP.



Exercise 1 — Arrays

Exercise 1: Array Min/Max

Write a Java program that declares an integer array of at least 8 elements and:

- 1 Traverses the array to find the **maximum** and **minimum** values
- 2 Prints both values with a descriptive message



Exercise 2 — Multi-Dimensional Arrays

Exercise 2: Matrix Sum

Create two 3×3 integer 2D arrays representing matrices A and B . Write a method that:

- 1 Computes their element-wise sum into a third matrix C
- 2 Prints C in tabular (row/column) format



Exercise 3 — Methods & Overloading

Exercise 3: Overloaded Area Calculator

Write an overloaded method `area()` that:

- 1 Returns the area of a square given one `int` side
- 2 Returns the area of a rectangle given two `double` sides
- 3 Returns the area of a circle given a `double` radius

Call all three versions from `main` and print the results.



Exercise 4: Factorial & Fibonacci

Write two recursive methods:

- 1 `factorial(int n)` that returns $n!$
- 2 `fibonacci(int n)` that returns the n -th Fibonacci number

Clearly identify the base case in each method, then print the first 10 terms produced by each.



Exercise 5 — OOP: Inheritance & Polymorphism

Exercise 5: Shape Hierarchy

Design a small class hierarchy:

- 1 Abstract class Shape with an abstract method area()
- 2 Subclasses Circle, Rectangle, Triangle that each **override** area()
- 3 In main, store several shapes in a Shape[] array and print each area (runtime polymorphism)



- GeeksforGeeks — *Object Oriented Programming (OOPs) Concept in Java*
- W3Schools — *Java Recursion*

Thank You!

Questions?

